

NetSage AI: Automated Network Diagnostic Platform

Comprehensive System Documentation

1. Executive Summary & Problem Statement

1.1 Problem Statement

In modern network management and educational lab environments (such as Cisco Packet Tracer), diagnosing multi-layer network failures requires extensive manual CLI execution (show commands), expert knowledge across OSI layers, and careful verification before executing remediation steps.

Key challenges include:

- **Time-Consuming Troubleshooting:** Engineers spend considerable time analyzing verbose CLI outputs to locate misconfigurations (e.g., sub-interface shutdown, incorrect wildcard masks, missing ACL entries).
- **Risk of Destructive Remediation:** Direct execution of AI-generated configuration commands in network environments introduces catastrophic risks if the model hallucinates or provides incorrect CLI parameters.
- **Lack of Structured Rules:** Pure LLM solutions lack deterministic guarantees, whereas traditional rule engines lack semantic reasoning capabilities.

1.2 System Objective

NetSage AI bridges this gap by combining **deterministic rule-based validation** with **structured prompt engineering** and a mandatory **Human-in-the-Loop (HITL) verification gate**.

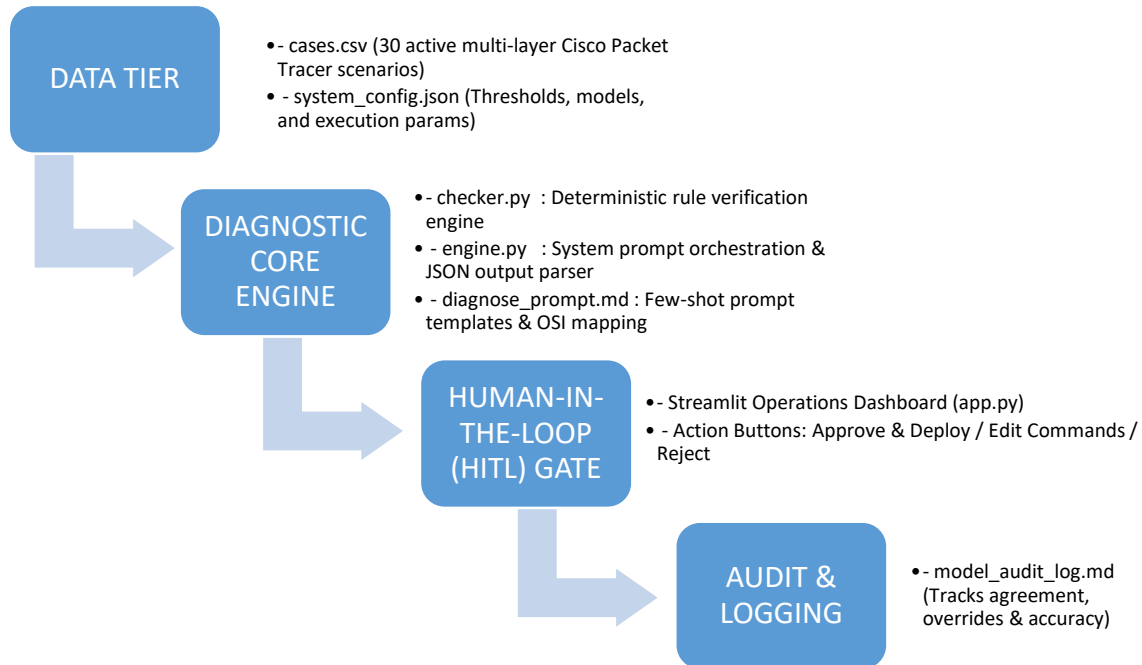
2. Proposed Solution

NetSage AI provides an end-to-end automated network diagnostic workspace:

1. **Hybrid Diagnostics:** Combines static pattern matching (checker.py) for known status errors with LLM prompt templates (diagnose_prompt.md) to extract root causes and evidence.
2. **Structured JSON Output:** Standardizes diagnostic results into strict fields: root_cause, osi_layer, confidence, evidence, next_command, and fix_steps.
3. **Human Oversight Gate:** Provides an interactive operations dashboard (app.py) where network engineers can inspect diagnoses, review evidence, edit proposed CLI commands, and approve or reject remediation before deployment.
4. **Auditability:** Logs agreement rates, false positives, and engineer overrides into a central model audit log (docs/model_audit_log.md).

3. Solution Architecture

NetSage AI follows a modular 4-tier architecture:



4. Detailed Design & Module Descriptions

File / Component	Responsibility
data/cases.csv	Dataset containing 30 structured test cases with attributes: case_id, symptom, topology_note, concept_tag, severity, show_outputs, and expected_fault.
prompts/diagnose_prompt.md	System prompt enforcing OSI layer identification, evidence extraction, few-shot examples, and strict JSON output schema.
src/checker.py	Deterministic engine containing regular expressions and rule checks (e.g., detecting administratively down, missing overload, mismatched VLAN IDs).

src/engine.py	Orchestrator module combining deterministic checks and prompt generation.
src/app.py	Interactive Streamlit dashboard providing case selection, real-time diagnostic visualizers, and human deployment approval controls.
docs/model_audit_log.md	Documentation tracking agreement metrics (76.6%) and edge cases requiring human override.

5. System Flowchart

Below is the workflow from case selection to human approval:

flowchart TD

```

A[Start: Operator Opens NetSage AI Dashboard] --> B[Load Dataset cases.csv]
B --> C[Select Case ID e.g., NET-001]
C --> D[Display Symptom, Topology Note & Show Outputs]
D --> E[Run Deterministic Rule Checker checker.py]
E --> F{Errors / Anomalies Detected?}
F -->|Yes| G[Flag Errors e.g., Sub-interface down]
F -->|No| H[Pass to Prompt Engine for LLM Inference]
G --> I[Format Structured JSON Diagnostic Output]
H --> I
I --> J[Display Diagnosis & Remediation CLI Commands on UI]
J --> K{Operator Decision Gate}
K -->|Approve & Deploy| L[Log Deployment Approval]
K -->|Edit Commands| M[Allow Operator CLI Override]
K -->|Reject| N[Flag False Positive in Audit Log]
L --> O[End: Action Recorded in System Log]
M --> O
N --> O

```

6. Primary Use Case

Use Case UC-01: Inter-VLAN Routing Diagnosis and Verification

- **Primary Actor:** Network Engineer / Student Lab Operator
- **Pre-conditions:** cases.csv loaded into Streamlit dashboard; Packet Tracer scenario context available.
- **Main Success Scenario:**
 1. The operator selects scenario NET-001 ("*PC1 cannot reach Server1 in VLAN 30*").
 2. NetSage AI parses the captured show_outputs:
GigabitEthernet0/0.10 is up, line protocol is up
GigabitEthernet0/0.30 is administratively down, line protocol is down
 3. The deterministic checker flags status: "ERRORS_DETECTED" and identifies GigabitEthernet0/0.30 as administratively down.
 4. NetSage AI presents the exact fix steps: ■ configure terminal
■ interface GigabitEthernet0/0.30
■ no shutdown
 5. The operator reviews the evidence, selects "**Approve & Deploy Fix**", and submits the decision.
- **Post-conditions:** The audit record is saved and the fix is approved for lab deployment.

7. Tech Stack Used

Category	Technology /Tool	Purpose
Language	Python 3.10+	Core development environment
User Interface	Streamlit	Building the interactive Operations Dashboard
Data Processing	Pandas	Loading, querying, and displaying structured CSV case data
Path Handling	pathlib	Dynamic, platform-agnostic file path resolution
Data Interchange	JSON	Structured prompt inputs and diagnostic outputs

Target System	Cisco IOS CLI / Packet Tracer	Target environment for network diagnostics
Documentation	Markdown & Mermaid.js	System architectural design, flowcharts, and audit logs